Analysis and Design of Algorithms Lab
Project Report

Ahmed Mohammed

May 2026

1 Problem Statement & Application Context

1.1 Real-World Problem

Competitive programming platforms such as Codeforces and LeetCode expose users to thousands of problems but provide no personalized guidance on what to solve next. As a result, users often waste limited study time on problems that are too easy, too difficult, or unrelated to their current weaknesses. This project proposes an algorithmic scheduling feature that automatically generates an optimal, personalized study session given a user's available time, skill level, and past performance.

1.2 Formal Problem Definition

Input:

- A problem set $P = \{p_1, p_2, \ldots, p_n\}$, where each problem p_i carries a unique ID, a difficulty rating, an estimated solving time t_i (minutes), and one or more topic tags.
- A user profile consisting of a skill level s and a proficiency map from the set of topics to $[0, 1]$.
- A history of previous attempts, each recording whether the problem was solved, the time taken, and the number of wrong submissions.
- A session time budget T (minutes).

Output: A subset $S \subseteq P$ such that

$$\sum_{p_i \in S} t_i \leq T \quad \text{and} \quad \sum_{p_i \in S} \text{benefit}(p_i, \text{user}) \text{ is maximized.}$$

,
where t_i is the time of problem i.
Benefit function. For a problem p and user u, the benefit is defined as:

$$\text{benefit}(p, u) = \underbrace{\bar{w}(p, u)}_{\text{topic weakness}} \times \underbrace{e^{-0.5\,|d_p - s|}}_{\text{difficulty match}} \times \underbrace{\frac{1}{1 + \text{attempts}(p, u)}}_{\text{recency penalty}} \times \underbrace{b(p, u)}_{\text{unsolved bonus}}$$

where $\bar{w}(p, u)$ is the mean weakness across p's topics $(1 - \pi(\text{topic}))$, d_p is the problem difficulty, s is the user's skill level, and $b(p, u) = 1.5$ if the user previously attempted but failed p, 0.5 if already solved, and 1.0 otherwise.

2 Algorithmic Approaches

2.1 Approach 1: Greedy

Design. Each problem is assigned a priority score equal to its benefit-to-time ratio $r_i = \text{benefit}(p_i, \text{user})/t_i$. Problems are sorted in descending order of r_i; ties are broken by preferring shorter problems. The scheduler then iterates through the sorted list and greedily adds each problem to the session as long as it fits within the remaining time budget.
Justification. The greedy ratio heuristic is the classical approach for the fractional knapsack problem, where it yields the exact optimum. The greedy algorithm actually performed very well in most test cases, compared to the second approach.
Complexity.

- Time: $O(n \log n)$ dominated by sorting; the linear scan is $O(n)$.
- Space: $O(n)$ for the rated-problem list.

Pros.

- Extremely fast.
- Memory space is negligible regardless of budget size.

Cons.

- Does not guarantee the globally optimal selection for 0-1 knapsack instances; it can leave large gaps of unused time.

2.2 Approach 2: Dynamic Programming

Design. Benefit values are real numbers; to apply integer DP they are scaled by a constant $C = 10^6$ and truncated to integers. A standard 0-1 knapsack DP table of size $(n+1) \times (T+1)$ is filled, where table$[i][t]$ stores the maximum scaled benefit achievable using the first i items with a time budget of t minutes. The selected problem set is recovered by backtracking through the table.
Justification. The 0-1 knapsack formulation models the problem exactly: each problem is either included or excluded, estimated times are positive integers, and benefit is additive. The DP solution is guaranteed to be globally optimal (up to the scaling precision).
Complexity.

- Time: $O(n \cdot T)$ where T is the session budget in minutes.
- Space: $O(n \cdot T)$ for the DP table (stored as a flat `long long` array to avoid 2-D allocation overhead).

Pros.

- Guarantees the optimal subset within the precision of the scaling constant.

Cons.

- Memory and runtime grow with T; large budgets produce large tables.
- Scaling floating-point benefits to integers introduces a small error proportional to $1/C$.
- Significantly slower than greedy for large inputs.

3 Situational Evaluation

Both algorithms solve the same problem but make different trade-offs between optimality and efficiency.

When greedy is preferable. In interactive or real-time settings: for instance, a mobile app that regenerates a session plan each time the user adjusts their time budget, the $O(n \log n)$ greedy approach responds instantly even for large problem sets. It is also preferable when the benefit values of individual problems are close to each other, as the ratio ranking tends to be near-optimal in that case. For typical competitive programming sessions (budget of 60–180 minutes, problem times of 10–150 minutes), the greedy solution empirically achieves benefit values within a few percent of the DP optimum.

When knapsack DP is preferable. When the scheduler is run as a batch or background job (e.g., generating a weekly study plan overnight), the runtime cost of DP is inconsequential, and the guaranteed optimality is worth it. DP also outperforms greedy noticeably when problem times are large and varied relative to the budget, creating scenarios where greedy's inability to "fill gaps" leads to suboptimal selections.

4 Sample Test Cases

The following cases illustrate how the scheduler behaves under different conditions. All cases use a time budget of $T = 124$ minutes.

Case 1 : New user, no history. A user with skill level 1200 and no attempt history. Because no recency penalty or unsolved bonus applies, benefit is driven purely by topic weakness and difficulty match. Both algorithms mostly agree on the same selection, since without history all benefits are smooth and the greedy ratio ranking tends to be near-optimal.

Case 2 : Experienced user with unsolved problems. A user with skill level 1200 and a history of 160 attempts, roughly half of which are unsolved. Problems previously attempted but not solved receive a 1.5× unsolved bonus, making them high-priority candidates. The knapsack DP exploits this by packing the session with as many high-bonus problems as fit.

Case 3 : Very short budget ($T = 20$ minutes). With a tight budget only the shortest high-benefit problems fit. The greedy approach may leave a few minutes unused when no remaining problem fits in the leftover time, while the DP always finds the globally optimal fill. The benefit gap between the two algorithms is most noticeable in this regime.

5 Performance Analysis

Table 1 and Figure 1 report measured runtimes and solution quality across combinations of problem-set size and user history size, with a fixed time budget of $T = 124$ minutes. Each runtime reading is the average of five independent runs, reported in microseconds (μs). The quality ratio column shows greedy total benefit as a percentage of knapsack total benefit; a value of 100% means both algorithms found equally optimal solutions.

Alignment with Theoretical Complexity

Greedy has time complexity $O(n \log n)$, dominated by sorting. This is visible in the data: as the problem count drops from 250 to 50, greedy runtime falls roughly proportionally. History size has

Table 1: Runtime and solution quality: greedy vs. knapsack DP

Problems	**History size**	**Knapsack runtime (μs)**	**Greedy runtime (μs)**	**Greedy/Knapsack benefit (%)**
250	55	695	621	99.65%
250	45	641	628	99.65%
250	30	1398	600	99.65%
250	10	539	445	97.60%
150	55	628	534	100.00%
150	45	677	1174	100.00%
150	30	707	439	100.00%
150	10	479	419	100.00%
50	55	229	179	100.00%
50	45	219	168	99.79%
50	30	208	150	99.79%
50	10	146	90	99.79%

negligible effect on greedy runtime because the benefit computation is $O(h)$ per problem (where h is history size), but h is small relative to the sort cost.

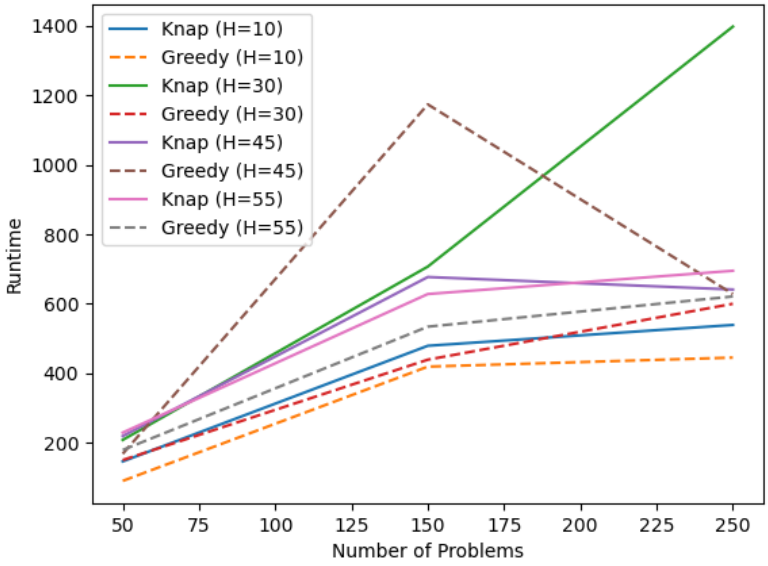

Figure 1: Runtime: greedy vs. knapsack DP

Knapsack DP has time complexity $O(n \cdot T)$, where T is the time budget. Since T is fixed at 120 minutes across all runs, the dominant variable is n. The measured runtimes decrease as n shrinks from 250 to 50, consistent with linear scaling in n. The weird spike at (250 problems, history 30, 1398 μs) is likely attributable to CPU cache effects rather than algorithmic behaviour, since the

surrounding rows at the same problem count are substantially lower.

Solution quality. The greedy algorithm matches the DP optimum in the majority of runs (100%), and never falls below 97.6% of the optimal benefit. This confirms that for this problem domain (moderate budget, small estimated times, smooth benefit distribution) greedy is a reliable approximation. The largest gaps appear at 250 problems with history size 10, where fewer previously-attempted problems exist, reducing the smoothing effect of the unsolved bonus and making the benefit landscape less uniform.

6 Reflection

6.1 AI Assistance

AI tooling was used in two specific areas of this project. First, the structure and components of the benefit function (topic weakness weighting, difficulty match score, recency penalty, and unsolved bonus) were developed with AI suggestions. Second, the input and output handling code (CSV parsing, file loading, and writing routines across `csv_utils`, `problem`, and `user`) was largely generated by AI and subsequently corrected and integrated by hand to match the project's architecture. All algorithmic logic (greedy selection and knapsack DP) was written independently.

6.2 Challenges

Modelling and file handling. A significant portion of development time was spent on system design: deciding how to represent problems, user profiles, and attempt history; how to persist and load them via CSV; and how the modules should interact. While file-handling code was largely generated with AI assistance, considerable effort went into reviewing, correcting, and integrating that code to match the intended architecture. The core algorithmic work could have been isolated and implemented much faster, but the goal was to build a realistic, end-to-end prototype rather than a standalone algorithm demo.

Handling floating-point values in DP. The benefit function produces real-valued scores, which are incompatible with the standard integer knapsack DP. The chosen solution multiplies every benefit by a large constant ($C = 10^6$) and truncates to an integer before filling the DP table. This preserves the relative ordering of problems with high precision while keeping the DP values bounded. The trade-off is a small quantisation error: two problems whose true benefits differ by less than $1/C$ may be treated as equal. This was deemed acceptable given that the benefit function itself is already a heuristic approximation.

6.3 Limitations

The prototype does not persist state between runs. User profiles and problem data are static CSV files generated offline by dedicated generator utilities (`gen_problems`, `gen_user`, `gen_test`). In a real deployment the system would require a database or API layer to track live solve events, update proficiency scores incrementally, and synchronise with the platform's problem set. Additionally, the benefit function is hand-crafted; a production system might learn its weights from historical data.